

Universitatea POLITEHNICA din București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

**Analiza imaginilor cu fund de ochi pentru ajutor în
diagnosticul retinopatiei diabetice**

Lucrare de dizertație

Prezentată ca cerință parțială pentru obținerea
titlului de *Master*
în domeniul *Calculatoare și Tehnologia Informației*
programul de studii *Ingineria Informației*

Conducător științific
Conf. dr. ing. Laura Maria FLOREA

Absolvent
Stancovici Marian

Anul 2026

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul *Analiza imaginilor cu fund de ochi pentru ajutor în diagnosticul retinopatiei diabetice*, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității “Politehnica” din București ca cerință parțială pentru obținerea titlului de *Master* în domeniul Inginerie Electronică și Telecomunicații/ Calculatoare și Tehnologia Informației, programul de studii *Ingineria Informației* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate. Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare. Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Iunie 2026.

Absolvent: Stancovici Marian

.....

Cuprins

Lista figurilor	iv
Lista tabelelor	v
Lista acronimelor	vii
1. Fundamente teoretice - context medical și tehnic	5
1.1. Diabetul zaharat și complicațiile sale oculare	5
1.2. Anatomia ochiului și retina	5
1.3. Retinopatia diabetică — definiție, stadii de evoluție și simptome	5
1.4. Imagistica de fund de ochi ("fundus photography")	5
1.5. Diagnosticul clinic — procesul actual și limitările sale	5
2. Metode existente pentru detecția automată a retinopatiei diabetice	6
2.1. Analiza seturilor de date disponibile	6
2.2. Metoda bazată pe arhitectura Inception-V3 [1]	8
2.3. "Transfer Learning" pentru diagnosticul retinopatiei diabetice [2]	8
2.4. "Multi-task Learning" pentru gradarea retinopatiei diabetice [3]	9
2.5. Analiza arhitecturii "Modified U-Net" pentru segmentarea semantică [4]	10
2.6. Tranziția către arhitecturi globale: RTNet (Relation Transformer Network) [5]	12
2.7. Arhitectura "Dual Branch" [6]	13
2.8. Abordarea "Explainable AI" [7]	14
2.9. Generalizarea în scenarii reale: "Multi-Model Domain Adaptation" [8]	14
2.10. Concluzii	15
Bibliografie	18
Anexa A. Cod	20

Lista figurilor

2.1. Exemple de imagini ale fundului de ochi în seturile de date DIARETDB0 și DIARETDB1 [7]	7
2.2. Tabel comparativ între seturile de date disponibile	7
2.3. Framework propus pentru detectarea și clasificarea retinopatiei diabetice [2] . . .	9
2.4. Arhitectura U-net modificată [4]	11
2.5. Curbele de Loss (stânga) și curbele Precision-Recall (dreapta) pentru studiile de ablație asupra celor patru leziuni DR. [5]	13

Lista tabelelor

Lista acronimelor

SOTA = State of the Art

Deep DRiD = Deep Diabetic Retinopathy Image Dataset

UoA-DR = University of Auckland Diabetic Retinopathy

IDRiD = Indian Diabetic Retinopathy Image Dataset

DDR = Dataset of Diabetic Retinopathy

DRIVE = Digital Retinal Images for Vessel Extraction

Introducere

Contextul și motivația lucrării

Retinopatia diabetică reprezintă o provocare majoră de sănătate publică, fiind principala cauză a orbirii prevenibile în rândul populației active. Creșterea continuă a numărului de pacienți cu diabet generează o presiune imensă asupra sistemelor medicale, depășind adesea capacitatea de screening a specialiștilor. Deoarece afecțiunea poate evolua asimptomatic în fazele inițiale, identificarea pacienților la risc este critică înainte ca leziunile retiniene să devină ireversibile, subliniind necesitatea unor soluții de monitorizare scalabile și eficiente.

Complexitatea diagnosticării este dată de necesitatea unei clasificări precise, conform standardelor internaționale care împart boala în cinci clase de severitate [3], [6]. Procesul impune medicului să distingă între stadiile incipiente, marcate doar de microanevrisme discrete [9], și formele avansate, caracterizate de hemoragii și neovascularizație. Această granularitate a diagnosticului este dificilă de menținut constant în programele de screening în masă, unde oboseala și subiectivitatea factorului uman pot duce la erori de clasificare cu impact direct asupra deciziilor terapeutice.

Pentru a depăși aceste limitări, domeniul imagisticii medicale a adoptat rapid soluțiile bazate pe inteligență artificială. Trecerea de la metodele clasice de procesare a imaginii [10] la algoritmi de învățare profundă (Deep Learning) a marcat un punct de inflexiune, validat clinic prin studii extensive [1]. Rețelele neurale convoluționale au eliminat necesitatea definirii manuale a regulilor vizuale [11], având capacitatea de a învăța autonom trăsături complexe direct din datele brute [7] și de a identifica corelații subtile între texturi și patologii.

Evoluția recentă a acestor tehnologii a permis tranziția de la modele experimentale simple la arhitecturi robuste [5], capabile să opereze în condiții clinice reale. Cercetarea actuală s-a distanțat de clasificarea binară elementară, concentrându-se pe sisteme care pot gestiona variabilitatea imaginilor provenite din diverse surse prin tehnici de adaptare la domeniu [8]. Algoritmi moderni demonstrează o reziliență crescută la artefacte și zgomot vizual, oferind o consistență a diagnosticului care rivalizează cu expertiza umană [7], [2].

Obiectivele lucrării

Scopul lucrării de față este de a folosi algoritmi de vedere computerizată bazați pe rețele convoluționale adânci pentru a crea o metodă de detectare și clasificare a retinopatiei diabetice. Metoda se va testa pe seturi de date publice. Implementarea se va face în Python folosind biblioteci specifice.

Aceasta urmărește atingerea următoarelor obiective:

- Analizarea metodelor deja propuse în literatura de specialitate pentru detecția și clasificarea automată a retinopatiei diabetice folosind imagini cu fund de ochi.

- Căutarea seturilor de date publice ce conțin imagini cu fund de ochi și adnotări pentru clasificarea retinopatiei diabetice.
- Analizarea seturilor de date găsite.
- Propunerea unei metode bazate pe învățare profundă pentru detecția și clasificarea retinopatiei diabetice.
- Analizarea performanței pentru metoda propusă în diverse cazuri de antrenare/testare

Structura lucrării

Lucrarea de față este structurată în mai multe capitole, fiecare din ele abordând un subiect esențial din proiect:

- Capitolul 1: Introducere

În acest capitol se prezintă contextul și motivația lucrării, cât și cum se propune să se redacteze și să se structureze, menționând aplicabilitatea acestuia în lumea reală.

- Capitolul 2: Fundamente teoretice - context medical și tehnic

Acest capitol va oferi o privire de ansamblu asupra retinopatiei diabetice, explicând cauzele, simptomele și stadiile acestei afecțiuni, precum și importanța detecției și clasificării automate a acestuia. De asemenea, se vor introduce conceptele de bază ale învățării profunde și ale rețelelor neuronale convoluționale, care vor fi utilizate în dezvoltarea metodei propuse.

- Capitolul 3: Metode existente pentru detecția automată a retinopatiei diabetice

Acest capitol va oferi o analiză detaliată a metodelor existente pentru detecția și clasificarea retinopatiei diabetice, evidențiind avantajele și limitările acestora, precum și modul în care acestea au evoluat în timp.

- Capitolul 4: Metodologie și implementare

Acest capitol va detalia metodologia și implementarea propusă pentru detecția și clasificarea retinopatiei diabetice, explicând arhitectura rețelei neuronale utilizate, procesul de antrenare și optimizare, precum și modul în care aceasta se diferențiază de metodele existente.

- Capitolul 5: Metoda propusă

Acest capitol va detalia metoda propusă pentru detecția și clasificarea retinopatiei diabetice, explicând arhitectura rețelei neuronale utilizate, procesul de antrenare și optimizare, precum și modul în care aceasta se diferențiază de metodele existente.

- Capitolul 6: Rezultate experimentale și discuții

Acest capitol va prezenta rezultatele obținute în urma testării metodei propuse pe seturile de date alese, comparând performanța acestuia cu alte metode existente și discutând avantajele și limitările sale.

- Capitolul 7: Concluzii

Acest capitol va sintetiza concluziile generale ale lucrării, evidențiind contribuțiile aduse și impactul potențial al metodei propuse în domeniul detecției automate a retinopatiei diabetice.

Capitolul 1

Fundamente teoretice - context medical și tehnic

- 1.1 Diabetul zaharat și complicațiile sale oculare
- 1.2 Anatomia ochiului și retina
- 1.3 Retinopatia diabetică — definiție, stadii de evoluție și simptome
- 1.4 Imagistica de fund de ochi ("fundus photography")
- 1.5 Diagnosticul clinic — procesul actual și limitările sale

Capitolul 2

Metode existente pentru detecția automată a retinopatiei diabetice

2.1 Analiza seturilor de date disponibile

Performanța și capacitatea de generalizare a modelelor de învățare profundă sunt intrinsec dependente de calitatea, volumul și diversitatea datelor utilizate în procesul de antrenare. În contextul oftalmologiei computaționale, disponibilitatea bazelor de date publice a permis tranziția de la metodele clasice de procesare a imaginii la arhitecturi neuronale complexe, oferind un teren comun pentru evaluarea obiectivă a algoritmilor.

- **Seturi de referință pentru clasificarea globală (“Grading”)**

- **EyePACS** (Kaggle Diabetic Retinopathy Detection) [1], [7] rămâne cea mai vastă resursă publică disponibilă, conținând aproximativ 88.702 de imagini. Importanța sa strategică constă în eterogenitatea extremă a imaginilor (variații de expunere, focalizare, artefacte), fiind critic pentru antrenarea unor rețele backbone robuste capabile să gestioneze fenomenul de “domain shift” [8] și să generalizeze pe imagini de calitate slabă
- **APTOS 2019** Blindness Detection [7] reprezintă standardul modern de calitate. Colectat de Aravind Eye Hospital, acest set de 3.662 de imagini este utilizat frecvent în literatura recentă pentru “fine-tuning”, oferind o vizibilitate superioară a detaliilor retiniene comparativ cu seturile mai vechi
- **MESSIDOR-2** [7] este considerat în prezent „Benchmark-ul de Aur” pentru testarea algoritmilor. Extinzând setul original Messidor la 1.748 de imagini, acesta se distinge prin consistența diagnostică și este utilizat ca domeniu țintă (Target Domain) în studiile de adaptare [8] și pentru validarea sistemelor explicabile (XAI) [7], oferind o metrică fidelă a performanței reale pe date curate.
- **DeepDRiD** [12] aduce o inovație necesară prin structurarea celor 2.000 de imagini în perechi binoculare și includerea notelor de calitate, simulând fluxul de triaj clinic.
- **UoA-DR** [7] este o resursă istorică (200 imagini), utilizată complementar în studiile comparative pentru a asigura diversitatea demografică a validării.

- **Seturi specializate pentru segmentarea leziunilor și analiza detaliată**

- **IDRiD** [7] este setul critic pentru sarcinile de segmentare semantică fină [5]. Pentru 81 dintre imagini, experții au trasat manual măști binare la nivel de pixel, făcându-l indispensabil pentru antrenarea arhitecturilor complexe de tip Transformer și pentru validarea localizării leziunilor în sistemele end-to-end [7].
- **DDR** [13] a revoluționat segmentarea pe scară largă, oferind 13.673 de imagini. Este esențial pentru antrenarea rețelelor adânci de segmentare (precum RTNet) fără a risca “overfitting-ul”, oferind adnotări “pixel-wise” pentru leziuni multiple [5].

- **E-Ophtha** [7] este o bază de date vitală pentru reducerea alarmelor false în metodele de detecție a exudatelor [10]. Valoarea sa unică constă în includerea imaginilor cu patologii non-diabetice care pot fi confundate cu DR
- **DiaRetDB1** [7] este standardul istoric pentru calibrarea sensibilității la leziuni roșii (microanevrisme) în lucrările de detecție automată [14]. Complementar, **HEI-MED** [15] oferă 169 de imagini focalizate exclusiv pe segmentarea zonelor de Edem Macular Diabetic.

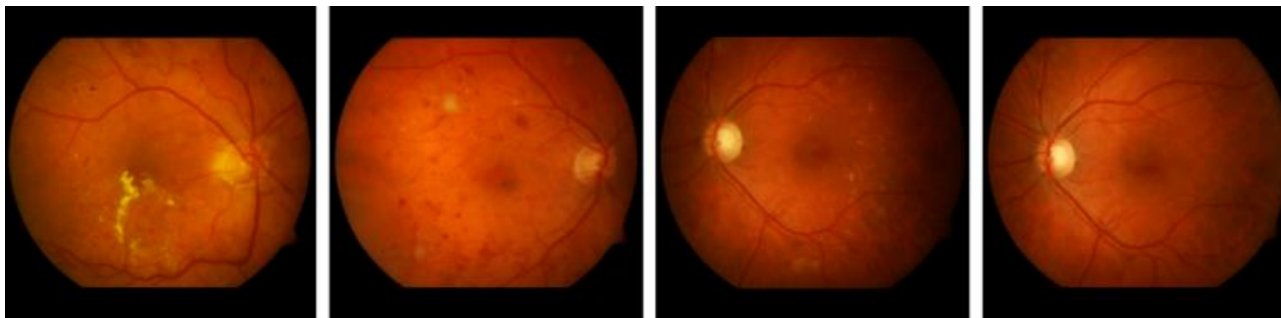


Figura 2.1: Exemple de imagini ale fundului de ochi în seturile de date DIARETDB0 și DIARETDB1 [7]

• Seturi standard pentru preprocesarea vasculară

- **DRIVE** [9] și **STARE** [7] sunt pilonii etapei de preprocesare. Deși conțin un număr redus de imagini, ele sunt utilizate obligatoriu pentru validarea algoritmilor de segmentare a vaselor de sânge, cum ar fi arhitecturile U-Net modificate [4] sau metodele clasice de extragere a trăsăturilor [14].

Tabelul de mai jos sintetizează caracteristicile tehnice esențiale ale seturilor de date analizate, evidențiind diferențele de volum, rezoluție și tipul de adnotare disponibil, elemente care dictează utilizarea lor în diversele etape ale proiectului.

Set de Date	Număr Imagini (Total)	Rezoluție Tipică (px)	Tip Adnotare (Label)	Utilizare Principală & Referință
EyePACS	88.702	Variabilă (până la 4000+)	Globală (Clase 0-4)	Antrenare Backbone, Generalizare
DDR	13.673	Variabilă	“Pixel-wise” (Măști) & BBox	Segmentare la scală largă
APTOS 2019	3.662	Înaltă (Standardizată)	Globală (Clase 0-4)	Fine-tuning, Validare
DeepDRiD	2.000	512x512 (Reconstruit)	Globală & Calitate & Binocular	Analiza calității imaginii
MESSIDOR-2	1.748	1440x960 - 2240x1488	Globală & Edem	Benchmark Testare
IDRiD	516	4288x2848	Pixel-wise (81 imgs) & Globală	Segmentare semantică fină
E-Ophtha	381 (EX) / 381 (MA)	Variabilă (Medii/Mari)	Contur Leziuni (EX/MA)	Reducere False Positive
UoA-DR	~200	Variabilă	Globală (Clase 0-4)	Validare încrucișată
HEI-MED	169	2196x1958	Măști Edem	Detecție Edem Macular
DiaRetDB1	89	1500x1152	Coordonate punctuale	Calibrare microanevrisme
DRIVE	40	565x584	Măști Vase (Binare)	Preprocesare (Vase de sange)
STARE	20	700x605	Măști Vase (Binare)	Preprocesare (vase de sange)

Figura 2.2: Tabel comparativ între seturile de date disponibile

2.2 Metoda bazată pe arhitectura Inception-V3 [1]

Studiul realizat de Gulshan et al. [1] este considerat un reper major în literatura de specialitate, marcând una dintre primele validări clinice riguroase ale utilizării rețelelor neuronale convoluționale pentru screening-ul automat al retinopatiei diabetice. Lucrarea demonstrează că un sistem de deep learning poate atinge și chiar depăși performanța medie a medicilor oftalmologi în detectarea retinopatiei diabetice, contribuind decisiv la acceptarea inteligenței artificiale în aplicații medicale reale [1].

Arhitectura utilizată este Inception-v3, o rețea convoluțională profundă caracterizată prin utilizarea modulelor Inception, care combină în paralel convoluții de dimensiuni diferite (1×1 , 3×3 , 5×5) și operații de pooling. Această strategie permite captarea informațiilor la multiple scări spațiale și reduce costul computațional prin utilizarea convoluțiilor 1×1 pentru diminuarea dimensionalității. Datorită acestei structuri, modelul este capabil să învețe reprezentări vizuale complexe, esențiale pentru identificarea leziunilor subtile specifice retinopatiei diabetice.

Modelul a fost antrenat pe un set de date extins, alcătuit din peste 128.000 de imagini fundus color, etichetate independent de mai mulți medici oftalmologi certificați în 2015. Utilizarea consensului între experți a contribuit la reducerea subiectivității etichetelor și la creșterea fiabilității procesului de antrenare. Sarcina principală a rețelei a fost clasificarea imaginilor în funcție de prezența sau absența retinopatiei diabetice referabile, conform standardelor clinice acceptate internațional [1].

Preprocesarea imaginilor a inclus decuparea automată a regiunii retiniene, redimensionarea la o rezoluție compatibilă cu Inception-v3 și normalizarea intensității pixelilor. În plus, au fost aplicate tehnici extensive de augmentare a datelor, precum rotații, translații și ajustări de luminozitate și contrast, pentru a îmbunătăți generalizarea și a reduce supraînvățarea [1] [2].

Evaluarea performanței a fost realizată pe seturile de date EyePACS-1 și Messidor-2 [7], unde modelul a obținut o arie sub curba ROC (AUC) de aproximativ 0.99 pentru detectarea retinopatiei diabetice referabile, cu valori ridicate de sensibilitate, comparabile cu performanța medicilor oftalmologi experimentați [1] [11].

2.3 “Transfer Learning” pentru diagnosticul retinopatiei diabetice [2]

Lucrarea propusă de Jabbar et al. [2] abordează problema diagnosticului automat al retinopatiei diabetice dintr-o perspectivă practică, concentrându-se pe utilizarea tehnicilor de “transfer learning” pentru a compensa limitările impuse de dimensiunea redusă a seturilor de date medicale etichetate. Studiul evidențiază faptul că modelele CNN pre-antrenate pe seturi de date generale de mari dimensiuni, precum ImageNet, pot fi adaptate eficient pentru analiza imaginilor fundus, obținând performanțe competitive cu un cost de antrenare semnificativ redus [2].

Autorii investighează mai multe arhitecturi consacrate de rețele neuronale convoluționale, printre care VGG16, VGG19, ResNet50, Inception-v3 și DenseNet, utilizate ca extractoare de caracteristici. Strategia de transfer learning constă în inițializarea rețelelor cu ponderi pre-antrenate, urmată de ajustarea straturilor superioare pentru sarcina specifică de clasificare a retinopatiei diabetice. În unele experimente, straturile convoluționale inferioare sunt înghețate pentru a preveni supraînvățarea, în timp ce în altele este aplicat “fine-tuning” progresiv pentru

a îmbunătăți adaptarea la domeniul imagisticii medicale. Una din abordările propuse poate fi observată în figura următoare.

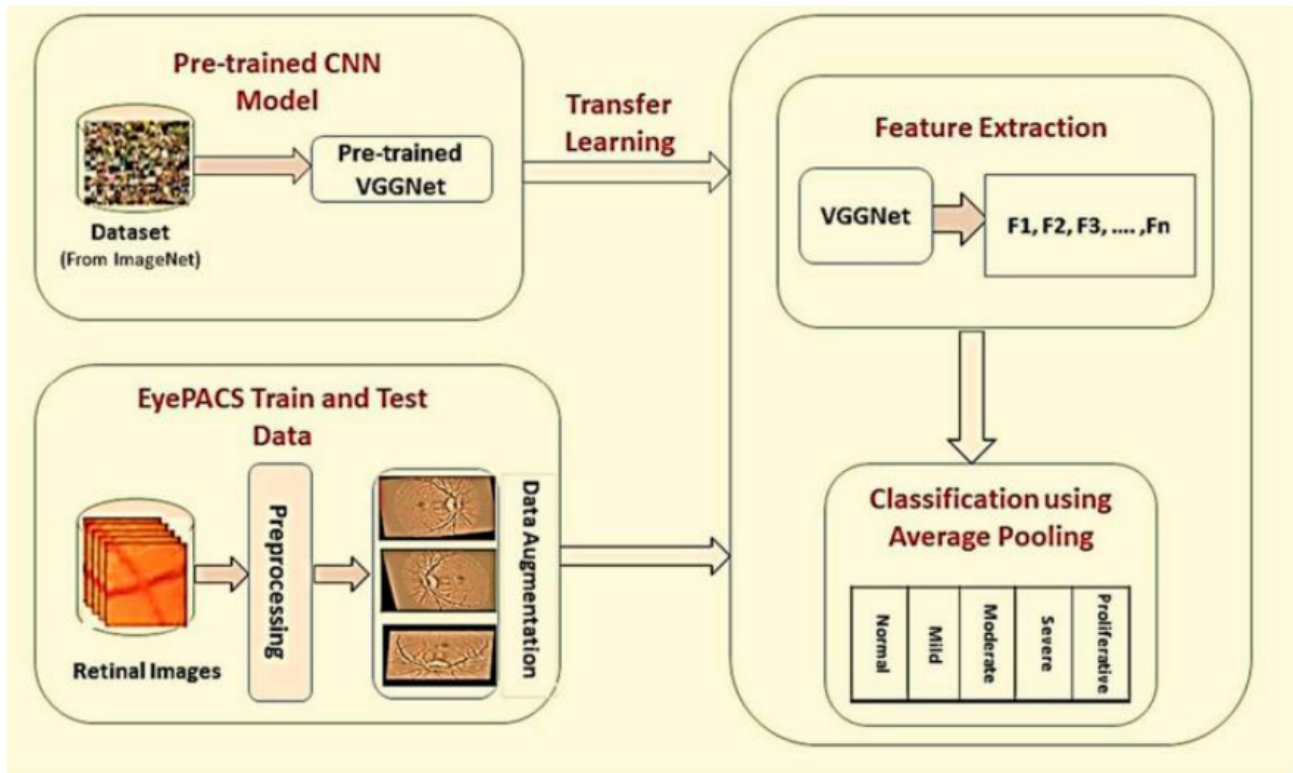


Figura 2.3: Framework propus pentru detectarea și clasificarea retinopatiei diabetice [2]

Seturile de date utilizate includ imagini fundus publice din Kaggle EyePACS [7], preprocesate prin redimensionare, normalizare și augmentare extensivă a datelor. Tehnicile de augmentare, precum rotațiile, flip-urile orizontale și ajustările de luminosități, joacă un rol central în creșterea diversității artificiale a datelor și în îmbunătățirea capacității de generalizare a modelelor. Această etapă este esențială în contextul “transfer learning-ului”, unde diferențele de distribuție dintre datele naturale și imaginile medicale pot afecta performanța rețelelor [2].

Evaluarea performanței este realizată folosind metrici standard, precum acuratețea, sensibilitatea, specificitatea și scorul F1, permițând o comparație riguroasă între diferitele arhitecturi analizate. Rezultatele experimentale indică faptul că modelele bazate pe DenseNet și ResNet obțin performanțe superioare față de arhitecturile mai vechi, confirmând avantajele reutilizării caracteristicilor vizuale generale în sarcini medicale specializate. Studiul subliniază că transfer learning-ul reprezintă o soluție viabilă și eficientă pentru implementarea rapidă a sistemelor automate de screening în contexte cu resurse limitate [2].

2.4 “Multi-task Learning” pentru gradarea retinopatiei diabetice [3]

Lucrarea propusă de Wang et al. [3] introduce o abordare de tip “deep multi-task learning” pentru gradarea automată a retinopatiei diabetice, având ca obiectiv principal exploatarea relației clinice dintre severitatea bolii și prezența leziunilor retiniene specifice. Spre deosebire de metodele clasice de clasificare “single-task”, autorii demonstrează că învățarea simultană a mai multor sarcini corelate clinic conduce la reprezentări vizuale mai discriminative și la performanțe superioare în sarcina principală de gradare [3].

Arhitectura propusă este alcătuită dintr-un backbone convoluțional comun, bazat pe o rețea CNN profundă (ResNet18), utilizat pentru extracția caracteristicilor globale din imaginile fundus. Peste acest extractor comun sunt construite mai multe ramuri specializate (task-specific branches), fiecare corespunzătoare unei sarcini distincte. Sarcina principală constă în clasificarea severității retinopatiei diabetice în mai multe stadii, în timp ce sarcinile auxiliare vizează detectarea prezenței diferitelor tipuri de leziuni, precum microanevrismele, exudatele dure, exudatele moi și hemoragiile, considerate “markeri” clinici relevanți ai progresiei bolii [3].

Un element central al arhitecturii îl reprezintă mecanismul de partajare controlată a caracteristicilor între sarcini. În locul unei separări rigide între ramuri, modelul permite propagarea informației relevante între sarcina de gradare și cele de detecție a leziunilor, încurajând învățarea unor reprezentări comune sensibile la semnalele patologice fine. Funcția de pierdere globală este definită ca o combinație ponderată a pierderilor individuale asociate fiecărei sarcini:

$$L_{isr} = \lambda_{img} \cdot L_{img_{isr}} + \lambda_{tv} \cdot L_{tv_{isr}} + \lambda_{sa} \cdot L_{seg-aware} + \lambda_{ca} \cdot L_{cls-aware}$$

,unde λ_{img} , λ_{tv} , λ_{sa} și λ_{ca} sunt hiperparametri care controlează contribuția pierderilor “intra-task”, “segmentation-aware” și “classification-aware”. Această formulare permite echilibrarea influenței fiecărei sarcini și stabilizează procesul de optimizare [3].

Modelul este antrenat și evaluat pe seturi de date publice de imagini fundus etichetate (DDR [13] și EyePACS [7]) atât cu stadiul retinopatiei diabetice, cât și cu adnotări la nivel de imagine privind prezența leziunilor. Imaginile sunt supuse unor etape standard de preprocesare, incluzând decuparea regiunii retiniene, redimensionarea și augmentarea datelor, pentru a reduce variațiile de iluminare și pentru a îmbunătăți capacitatea de generalizare. Prin includerea sarcinilor auxiliare, modelul beneficiază de un efect de regularizare implicită, reducând riscul de supraînvățare, aspect observat și în lucrările anterioare bazate pe CNN-uri “single-task” [1], [11].

Rezultatele experimentale raportate în articol indică îmbunătățiri consistente ale performanței față de modelele “single-task”, atât în termeni de acuratețe, cât și de scor F1 pentru sarcina de gradare a retinopatiei diabetice. În plus, autorii evidențiază o creștere a stabilității predicțiilor și o sensibilitate mai bună la stadiile intermediare ale bolii, considerate dificil de clasificat. Aceste rezultate confirmă faptul că integrarea informațiilor despre leziuni în procesul de antrenare contribuie direct la o evaluare mai precisă a severității retinopatiei diabetice și validează eficiența paradigmei “multi-task learning” în context clinic [3].

2.5 Analiza arhitecturii “Modified U-Net” pentru segmentarea semantică [4]

Segmentarea semantică a structurilor retiniene, în special a rețelei vasculare și a leziunilor asociate retinopatiei diabetice, reprezintă o etapă critică în diagnosticarea automată, deoarece modificările morfologice ale vaselor de sânge sunt adesea primii indicatori ai bolii. Deși arhitectura UNet, introdusă inițial de Ronneberger et al. [16] pentru segmentarea biomedicală, a devenit standardul de aur în domeniu datorită structurii sale simetrice de tip “encoder-decoder”, aceasta prezintă limitări în detectarea vaselor capilare foarte fine, pierzând informații spațiale esențiale în procesul de subeșantionare.

În studiul “Modified U-Net architecture for semantic segmentation of diabetic retinopathy images”, Sambyal et al [4]. propun o optimizare arhitecturală semnificativă pentru a contracara

aceste deficiențe. Modelul propus, denumit Modified U-Net, păstrează structura în formă de „U” a rețelei originale, dar inovează fundamental la nivelul blocurilor de procesare.

Autorii au înlocuit blocurile convoluționale standard cu blocuri reziduale (residual blocks), inspirate din arhitecturile de tip ResNet. Această modificare este crucială din punct de vedere tehnic: în rețelele adânci, apare frecvent problema dispariției gradientului (vanishing gradient), ceea ce face antrenarea dificilă și duce la pierderea detaliilor fine. Prin introducerea conexiunilor reziduale (“skip connections” la nivel de bloc), informația originală este propagată mai eficient prin straturi, permițând rețelei să învețe caracteristici complexe fără a degrada performanța pe detaliile de finețe, precum microanevrismele sau vasele subțiri. Aceste detalii se pot observa și în figura următoare.

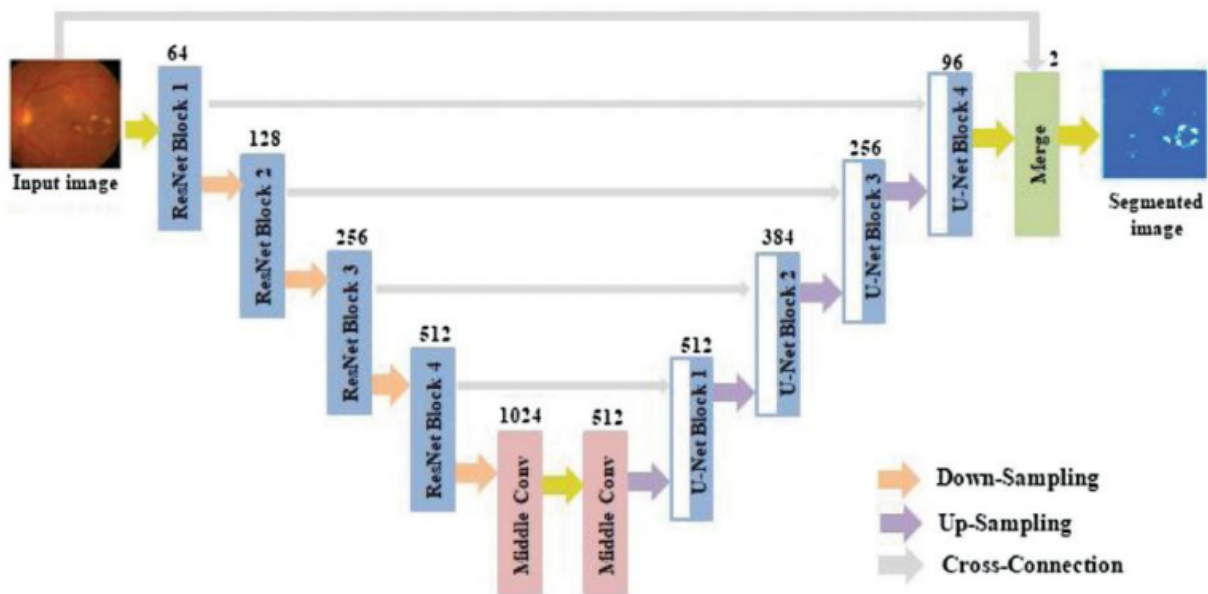


Figura 2.4: Arhitectura U-net modificată [4]

Importanța acestei abordări constă în capacitatea de generalizare comparativ cu metodele clasice de procesare a imaginii, precum cele bazate pe “clustering fuzzy” propuse anterior de Sopharak et al. [10]. În timp ce metodele de tip “clustering” sunt sensibile la zgomot și la variațiile de iluminare (frecvente în imaginile de fund de ochi), arhitectura Modified U-Net demonstrează o robustețe sporită prin învățarea contextuală a trăsăturilor

Pentru validarea performanței, metoda a fost testată pe seturile de date publice standardizate DRIVE [9] și STARE [7], recunoscute pentru dificultatea lor în segmentarea vaselor.

Rezultatele experimentale raportate de Sambyal et al. [4] au indicat o îmbunătățire clară față de U-Net-ul standard. Pe setul de date DRIVE, autorii au obținut o acuratețe maximă de 96.94 sub curba ROC (AUC) de 0.98, valori care plasează metoda în topul soluțiilor de segmentare. Mai mult, analiza vizuală a rezultatelor arată o continuitate mult mai bună a vaselor de sânge segmentate și o reducere a alarmelor false în zonele cu patologii, demonstrând că integrarea învățării reziduale în arhitectura de segmentare este o direcție esențială pentru analiza detaliată a retinopatiei diabetice.

2.6 Tranziția către arhitecturi globale: RTNet (Relation Transformer Network) [5]

Dacă modelele bazate pe U-Net, precum cel discutat anterior [4], excelează în capturarea detaliilor locale, ele suferă totuși de o limitare inerentă a rețelelor convoluționale: incapacitatea de a modela dependențe la distanță mare (long-range dependencies). În patologia retiniană, leziunile nu apar izolat; există adesea o corelație spațială între poziția vaselor de sânge și apariția microanevrismelor sau a exudatelor. Pentru a adresa această lipsă de „context global”, Huang et al. [5] au propus în 2022 o schimbare de paradigmă prin introducerea RTNet (Relation Transformer Network), o arhitectură care marchează trecerea către mecanismele de atenție.

Inovația centrală a RTNet constă în înlocuirea parțială a operațiilor convoluționale standard cu blocuri de tip “Transformer”, denumite Relation Transformer Blocks (RTB). Spre deosebire de CNN-uri care „văd” doar o fereastră mică de pixeli la un moment dat, RTB-ul utilizează două mecanisme de atenție complementare:

1. “Self-Attention” (Atenție proprie): Permite modelului să înțeleagă relațiile dintre leziunile dispersate pe toată suprafața retinei (de exemplu, grupuri de exudate dure care semnalează o zonă de edem).
2. “Cross-Attention” (Atenție încrucișată): Autorii au observat că leziunile vasculare sunt strâns legate de structura vaselor de sânge. Astfel, mecanismul de “cross-attention” folosește harta vaselor de sânge ca un „ghid” pentru a localiza mai precis leziunile fine, transferând informație structurală dinspre ramura vasculară către ramura de detectare a leziunilor.

Pentru validare, metoda a fost testată pe seturile de date IDRiD [7] și DDR [13], care conțin adnotări la nivel de pixel pentru multiple tipuri de leziuni. Rezultatele cantitative au demonstrat că RTNet depășește metodele existente anterioare (inclusiv diverse variante de U-Net și DeepLab), obținând cele mai bune scoruri de segmentare pentru Exudate (EX), Microanevrisme (MA) și Exudate Moi (SE).

Deși inovația teoretică a arhitecturii RTNet (Relation Transformer Network) propusă de Huang et al. [5] este evidentă prin integrarea mecanismelor de atenție, validarea practică a acesteia necesită o demonstrație riguroasă a faptului că fiecare modul propus contribuie activ la îmbunătățirea segmentării. În acest sens, autorii au realizat un studiu extensiv de ablație (ablation study), analizând impactul incremental al componentelor arhitecturale asupra capacității de învățare și generalizare a rețelei.

Această analiză comparativă este sintetizată în graficele de performanță care monitorizează evoluția funcției de pierdere (Loss) și curbele Precision-Recall (PR) pentru cele patru tipuri principale de leziuni asociate retinopatiei diabetice: microanevrisme (MA), exudate (EX), hemoragii (HE) și exudate moi (SE).

Comparatia pornește de la un model “Baseline” (o rețea convoluțională standard de tip U-Net fără mecanisme de atenție) și adaugă progresiv componentele specifice RTNet, regasite în figura de mai jos:

1. GTB (Global Transformer Block): Adăugarea capacității de procesare globală.
2. SAH (Self-Attention Head): Introducerea atenției proprii pentru corelarea leziunilor similare.

3. RAH (Relation Attention Head / Cross-Attention): Modulul critic care leagă leziunile de structura vaselor de sânge.
4. RTB (Relation Transformer Block): Arhitectura completă propusă.

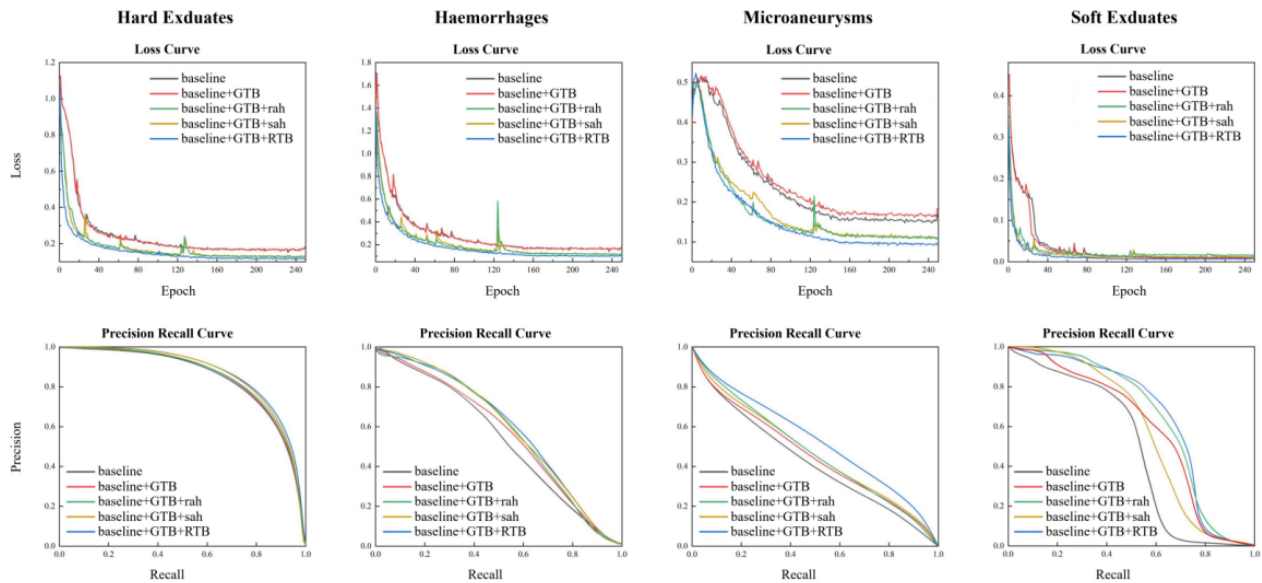


Figura 2.5: Curbele de Loss (stânga) și curbele Precision-Recall (dreapta) pentru studiile de ablație asupra celor patru leziuni DR. [5]

Lucrarea lui Huang et al. [5] validează ipoteza că integrarea informației globale prin Transformere este pasul necesar pentru a depăși plafonul de performanță atins de rețelele convoluționale pure în segmentarea medicală complexă.

2.7 Arhitectura “Dual Branch” [6]

Deși progresele în segmentarea semantică, ilustrate prin metodele bazate pe U-Net [4] sau pe Transformere [5], au permis o delimitare precisă a leziunilor, translatarea acestor detalii într-un diagnostic global (grading) rămâne o provocare majoră din cauza naturii subtile a diferențelor dintre stadiile retinopatiei diabetice. Rețelele convoluționale standard utilizate în studiile fundamentale, precum cel al lui Gulshan et al. [1], tind să piardă detaliile fine, cum ar fi microanevrismele incipiente, în procesul de redimensionare a imaginii necesar pentru analiza globală. Pentru a depăși acest compromis inherent între contextul vizual și rezoluția detaliilor, Shakibania et al. [6] au propus o arhitectură inovatoare de tip “Dual Branch Deep Learning Network”, care restructurează procesul de învățare prin decuplarea extragerii trăsăturilor în două fluxuri paralele distincte.

Conceptul central al acestei abordări constă în imitarea procesului cognitiv uman de diagnosticare, procesând imaginea simultan pe o „ramură globală” și o „ramură locală”. Ramura globală analizează imaginea completă a fundului de ochi pentru a captura structura generală a vaselor și geometria retinei, o strategie similară cu metodele clasice de “Transfer Learning” [2]. În paralel, ramura locală operează pe decupaje de înaltă rezoluție, fiind specializată în identificarea leziunilor microscopice care ar fi invizibile pentru rețeaua globală. Elementul de noutate adus de această lucrare față de abordările “multi-task” anterioare [3], este mecanismul de fuziune a celor două fluxuri, care utilizează tehnici de atenție pentru a pondera importanța

trăsăturilor locale în decizia finală, asigurând că un detaliu critic nu este ignorat în favoarea unei imagini de ansamblu aparent sănătoase.

Eficiența acestei arhitecturi hibride este validată prin rezultatele experimentale obținute pe seturile de date standard, unde modelul “Dual Branch” a demonstrat o superioritate clară în clasificarea stadiilor intermediare ale bolii (ușor și moderat). Analiza performanței indică faptul că integrarea informației locale crește semnificativ sensibilitatea modelului și reduce confuzia între clasele adiacente, oferind o robustețe superioară la variațiile de calitate a imaginii comparativ cu arhitecturile “singlestream”. Astfel, lucrarea confirmă faptul că performanța actuală în 2024 nu derivă doar din adâncimea rețelei, ci din capacitatea arhitecturală de a reuni viziunea macroscopică cu cea microscopică într-un diagnostic coerent.

2.8 Abordarea “Explainable AI” [7]

În timp ce arhitecturile precum “Dual Branch Network” [6] sau Transformer-ele relaționale [5] au reușit să împingă limitele acurateței și ale segmentării fine, integrarea acestor sisteme în fluxul clinic real se lovește de o barieră fundamentală: lipsa de încredere a medicilor în algoritmii de tip „black-box”. Validarea clinică inițiată de Gulshan et al. [1] a demonstrat potențialul diagnostic, însă pentru ca un sistem de inteligență artificială să fie acceptat ca asistent de încredere, acesta trebuie să își poată justifica deciziile. În acest context, lucrarea publicată de Chetoui și Akhloufi [7], reprezintă un pilon esențial al stadiului actual al cunoașterii, adresând simultan două probleme critice: interpretabilitatea (Explainable AI - XAI) și robustețea pe seturi de date eterogene.

Metodologia propusă de autori se bazează pe o arhitectură de tip CNN modificată (utilizând backbone-uri precum DenseNet121 sau ResNet50), în care straturile clasice complet conectate sunt înlocuite cu Global Average Pooling (GAP). Această modificare tehnică permite generarea Hărților de Activare a Claselor (CAM), oferind medicului o confirmare vizuală a regiunilor patologice care au determinat diagnosticul.

Relevanța clinică a acestei abordări extensive este evidențiată în rezultatele vizuale, unde hărțile de saliență generate de algoritm se suprapun cu precizie peste semnele patologice reale, indiferent de setul de date din care provine imaginea. Zonele marcate cu roșu intens pe hărțile CAM corespund anatomic cu microanevrismele și exudatele, confirmând că rețeaua neuronală a învățat trăsăturile intrinseci ale bolii și nu caracteristicile specifice ale camerei foto (bias-ul echipamentului). Prin urmare, modelul propus [7] validează faptul că un sistem trebuie să fie nu doar precis, ci și independent de sursa datelor și transparent în decizii.

2.9 Generalizarea în scenarii reale: “Multi-Model Domain Adaptation” [8]

Ultima frontieră în dezvoltarea sistemelor de diagnostic asistat de calculator (CAD) pentru retinopatia diabetică nu mai este legată strict de acuratețea obținută pe un singur set de date de antrenament, ci de capacitatea algoritmului de a funcționa corect într-un mediu clinic eterogen. În realitate, imaginile de fund de ochi variază drastic în funcție de modelul camerei, setările de iluminare, etnia pacientului sau protocolul de achiziție, fenomen cunoscut sub numele de “domain shift”. Modelele antrenate pe un „domeniu sursă” (de exemplu, imagini de înaltă calitate de la EyePACS [7]) tind să aibă performanțe scăzute atunci când sunt aplicate pe un „domeniu țintă” (imagini clinice noi, cu caracteristici vizuale diferite), ceea ce limitează sever aplicabilitatea lor universală.

Pentru a soluționa această problemă critică de implementare, Zhang et al. [8] au propus în 2022 o metodă inovatoare denumită “Multi-Model Domain Adaptation” (MMDA). Spre deosebire de abordările anterioare care încearcă să alinieze domeniile folosind un singur model, autorii introduc o arhitectură complexă bazată pe o rețea siameză, utilizând două modele distincte pentru a extrage trăsături complementare și pentru a crește robustețea deciziei.

Inovația tehnică centrală a lucrării constă în integrarea simultană a trei mecanisme de optimizare care lucrează concertat pentru a forța modelul să ignore diferențele stilistice dintre seturile de date și să se concentreze exclusiv pe patologie. Astfel, arhitectura combină o pierdere de clasificare pentru a asigura acuratețea diagnostică pe domeniul sursă etichetat, cu o pierdere de discrepanță care maximizează diferența dintre predicțiile celor două clasificatoare pe domeniul țintă pentru a detecta exemplele aflate la granița de decizie. În plus, se utilizează o pierdere de adaptare adversarială care, prin intermediul unui discriminator, minimizează distanța dintre distribuția trăsăturilor din sursă și cea din țintă, făcând ca imaginile provenite din spitale diferite să fie reprezentate similar de către rețeaua neuronală.

Validarea experimentală a metodei a fost realizată folosind setul de date EyePACS [7] ca domeniu sursă și Messidor-2 [7] ca domeniu țintă, simulând un scenariu realist de transfer tehnologic de la imagini de screening la imagini clinice de referință. Rezultatele au demonstrat că abordarea MMDA depășește semnificativ metodele standard de antrenare și alte tehnici de adaptare cunoscute, obținând o îmbunătățire substanțială a metricilor de performanță pe domeniul țintă fără a necesita etichete pentru acesta. Această lucrare încheie logic analiza stadiului actual al cunoașterii, subliniind faptul că un sistem complet nu trebuie doar să segmenteze leziuni fine sau să fie explicabil, ci trebuie să fie capabil să iasă din mediul controlat al laboratorului și să ofere o performanță robustă și generalizabilă în diversele condiții întâlnite în practica medicală globală.

2.10 Concluzii

Analiza literaturii de specialitate relevă faptul că detectarea automată a retinopatiei diabetice a depășit stadiul de experiment teoretic, devenind o direcție de cercetare matură, cu rezultate clinice valide. Totuși, problema rămâne una de o complexitate ridicată, care depășește simpla clasificare a imaginilor. Dificultatea majoră nu constă doar în volumul masiv de date necesar, ci în subtilitatea semnelor patologice incipiente, precum microanevrismele, care pot ocupa doar câțiva pixeli dintr-o imagine de înaltă rezoluție, fiind ușor de confundat cu artefactele sau zgomotul de fond. Această granularitate fină impune algoritmilor o capacitate de discriminare vizuală care, în anumite scenarii limită, trebuie să depășească acuitatea ochiului uman.

În ciuda acestor obstacole, soluțiile algoritmice existente demonstrează o evoluție remarcabilă. Trecerea de la rețelele convoluționale standard la arhitecturi sofisticate, capabile să integreze informația globală cu cea locală prin mecanisme de tip “dual-branch” [6], a permis o creștere semnificativă a sensibilității în detectarea stadiilor intermediare ale bolii. Mai mult, rezultatele obținute prin tehnici de adaptare la domeniu [8] sugerează că bariera variabilității echipamentelor medicale începe să fie depășită, deschizând calea pentru sisteme robuste, capabile să funcționeze corect indiferent de spitalul în care sunt implementate.

Un alt aspect promițător este schimbarea de paradigmă de la modelele de tip „cutie neagră” la sistemele explicabile (Explainable AI). Așa cum evidențiază studiile recente [7], capacitatea modelelor de a genera hărți de activare precise transformă inteligența artificială dintr-un simplu clasificator statistic într-un partener transparent pentru medic, facilitând validarea rapidă a

deciziei. Deși provocările legate de dezechilibrul claselor și de generalizarea pe populații diverse persistă, performanțele atinse de metodele actuale, validate pe seturi de date riguroase precum MESSIDOR-2 [7], confirmă potențialul acestor tehnologii de a eficientiza screening-ul în masă și de a preveni orbirea la scară globală.

Bibliografie

- [1] Varun Gulshan, Lily Peng, Marc Coram, Martin C Stumpe, Derek Wu, Arunachalam Narayanaswamy, Subhashini Venugopalan, Kasumi Widner, Tom Madams, Jorge Cuadros, et al. Development and validation of a deep learning algorithm for detection of diabetic retinopathy in retinal fundus photographs. *jama*, 316(22):2402–2410, 2016.
- [2] Muhammad Kashif Jabbar, Jianzhuo Yan, Hongxia Xu, Zaka Ur Rehman, and Ayesha Jabbar. Transfer learning-based model for diabetic retinopathy diagnosis using retinal images. *Brain Sciences*, 12(5):535, 2022.
- [3] Xiaofei Wang, Mai Xu, Jicong Zhang, Lai Jiang, and Liu Li. Deep multi-task learning for diabetic retinopathy grading in fundus images. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 2826–2834, 2021.
- [4] Nitigya Sambyal, Poonam Saini, Rupali Syal, and Varun Gupta. Modified u-net architecture for semantic segmentation of diabetic retinopathy images. *Biocybernetics and Biomedical Engineering*, 40(3):1094–1109, 2020.
- [5] Shiqi Huang, Jianan Li, Yuze Xiao, Ning Shen, and Tingfa Xu. Rtnet: relation transformer network for diabetic retinopathy multi-lesion segmentation. *IEEE Transactions on Medical Imaging*, 41(6):1596–1607, 2022.
- [6] Hossein Shakibania, Sina Raoufi, Behnam Pourafkham, Hassan Khotanlou, and Muharram Mansoorizadeh. Dual branch deep learning network for detection and stage grading of diabetic retinopathy. *Biomedical Signal Processing and Control*, 93:106168, 2024.
- [7] Mohamed Chetoui and Moulay A Akhloufi. Explainable end-to-end deep learning for diabetic retinopathy detection across multiple datasets. *Journal of Medical Imaging*, 7(4):044503–044503, 2020.
- [8] Guanghua Zhang, Bin Sun, Zhaoxia Zhang, Jing Pan, Weihua Yang, and Yunfang Liu. Multi-model domain adaptation for diabetic retinopathy classification. *Frontiers in Physiology*, 13:918929, 2022.
- [9] Joes Staal, Michael D Abràmoff, Meindert Niemeijer, Max A Viergever, and Bram Van Ginneken. Ridge-based vessel segmentation in color images of the retina. *IEEE transactions on medical imaging*, 23(4):501–509, 2004.
- [10] Akara Sopharak, Bunyarit Uyyanonvara, and Sarah Barman. Automatic exudate detection from non-dilated diabetic retinopathy retinal images using fuzzy c-means clustering. *sensors*, 9(3):2148–2161, 2009.
- [11] Harry Pratt, Frans Coenen, Deborah M Broadbent, Simon P Harding, and Yalin Zheng. Convolutional neural networks for diabetic retinopathy. *Procedia computer science*, 90:200–205, 2016.
- [12] Ruhan Liu, Xiangning Wang, Qiang Wu, Ling Dai, Xi Fang, Tao Yan, Jaemin Son, Shiqi Tang, Jiang Li, Zijian Gao, et al. Deepdrid: Diabetic retinopathy—grading and image quality estimation challenge. *Patterns*, 3(6), 2022.

- [13] Tao Li, Yingqi Gao, Kai Wang, Song Guo, Hanruo Liu, and Hong Kang. Diagnostic assessment of deep learning algorithms for diabetic retinopathy screening. *Information Sciences*, 501:511–522, 2019.
- [14] Meindert Niemeijer, Bram Van Ginneken, Joes Staal, Maria SA Suttorp-Schulten, and Michael D Abràmoff. Automatic detection of red lesions in digital color fundus photographs. *IEEE Transactions on medical imaging*, 24(5):584–592, 2005.
- [15] Luca Giancardo, Fabrice Meriaudeau, Thomas P Karnowski, Yaqin Li, Seema Garg, Kenneth W Tobin Jr, and Edward Chaum. Exudate-based diabetic macular edema detection in fundus images using publicly available datasets. *Medical image analysis*, 16(1):216–226, 2012.
- [16] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*, pages 234–241. Springer, 2015.

Anexa A

Cod

```

1 # =====
2 # IMPORTURI SI ELIBERARE MEMORIE
3 # =====
4 import os
5 import time
6 import gc
7 import pandas as pd
8 import numpy as np
9 import torch
10 import torch.nn as nn
11 import torch.optim as optim
12 from torch.utils.data import Dataset, DataLoader
13 from torchvision import transforms, models
14 from PIL import Image
15 from sklearn.metrics import accuracy_score, f1_score
16 import matplotlib.pyplot as plt
17
18 def curata_memoria():
19     print("Initializare proces de curatare a memoriei...")
20
21     # 1. Identificarea si stergerea variabilelor globale reziduale
22     variabile_grele = ['model', 'train_loader', 'test_loader', 'train_ds', 'test_ds',
23                        'full_dataset', 'images', 'labels', 'optimizer', 'df']
24
25     pentru_curatare = 0
26     for var in variabile_grele:
27         if var in globals():
28             del globals()[var]
29             pentru_curatare += 1
30
31     # 2. Eliberare memorie RAM
32     gc.collect()
33
34     # 3. Eliberare memorie VRAM si procese IPC ramase de la dataloadere
35     if torch.cuda.is_available():
36         torch.cuda.empty_cache()
37         torch.cuda.ipc_collect()
38
39     print(f"S-au eliminat {pentru_curatare} variabile din mediul global.")
40     print("Memoria RAM si VRAM a fost eliberata cu succes.\n")
41
42 curata_memoria()
43
44 # =====
45 # 0. CONFIGURATII EXPERIMENTE (DEEP FINE-TUNING)
46 # =====
47 EXPERIMENTE = [
48     # --- Partea 1: Arhitecturi diferite evaluate pe setul APTOS ---
49     {
50         "nume": "Exp1_DenseNet121_DeepFT_Aptos",
51         "model": "densenet121",
52         "train_datasets": ["APTOS"],
53         "test_datasets": ["APTOS"],
54         "epoci": 20, "aug": True, "fine_tune_deep": True, "use_weighted_loss": False
55     },
56     {
57         "nume": "Exp2_EfficientNetB0_DeepFT_Aptos",
58         "model": "efficientnet_b0",
59         "train_datasets": ["APTOS"],
60         "test_datasets": ["APTOS"],
61         "epoci": 20, "aug": True, "fine_tune_deep": True, "use_weighted_loss": False
62     },
63 ]

```

```

64 # --- Partea 2: Evaluare model de baza (ResNet50) pe seturi de date eterogene ---
65 # Setul IDRID: Volum redus de date, predispozitie la overfitting
66 {
67     "nume": "Exp3_ResNet50_DeepFT_IDRID",
68     "model": "resnet50",
69     "train_datasets": ["IDRID"],
70     "test_datasets": ["IDRID"],
71     "epoci": 25,
72     "aug": True, "fine_tune_deep": True, "use_weighted_loss": False
73 },
74 # Setul MESSIDOR: Volum mediu de date, echipament de achizitie diferit
75 {
76     "nume": "Exp4_ResNet50_DeepFT_MESSIDOR",
77     "model": "resnet50",
78     "train_datasets": ["MESSIDOR"],
79     "test_datasets": ["MESSIDOR"],
80     "epoci": 20, "aug": True, "fine_tune_deep": True, "use_weighted_loss": False
81 },
82 # Setul EYEPACS: Volum extins de date, variabilitate mare
83 {
84     "nume": "Exp5_ResNet50_DeepFT_EYEPACS",
85     "model": "resnet50",
86     "train_datasets": ["EYEPACS"],
87     "test_datasets": ["EYEPACS"],
88     "epoci": 10,
89     "aug": False,
90     "fine_tune_deep": True, "use_weighted_loss": False
91 },
92 {
93     "nume": "Exp6_ResNet50_DeepFT_EYEPACS_Aug",
94     "model": "resnet50",
95     "train_datasets": ["EYEPACS"],
96     "test_datasets": ["EYEPACS"],
97     "epoci": 10,
98     "aug": True,
99     "fine_tune_deep": True, "use_weighted_loss": False
100 }
101 ]
102
103 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
104 print(f"Device curent: {device} | Numar GPU-uri detectate: {torch.cuda.device_count()}")
105
106 # =====
107 # 1. CONFIGURARE CAI DIRECTOARE
108 # =====
109 def get_dataset_paths(dataset_name):
110     """ Returneaza caile catre fisierele CSV si directoarele cu imagini aferente fiecarui set.
111         """
112     if dataset_name == "APTOS":
113         base = "/home/marian-s/Disertatie/Dataset_APTOS_2019"
114         return {"csv": f"{base}/train_1.csv", "dir": f"{base}/train_images/train_images"}
115
116     elif dataset_name == "EYEPACS":
117         base = "/home/marian-s/Disertatie/Dataset_KaggleEyePACS"
118         return {"csv": f"{base}/labels/traintestLabels15_trainLabels19.csv", "dir": f"{base}/resized_traintest15_train19"}
119
120     elif dataset_name == "IDRID":
121         base = "/home/marian-s/Disertatie/Dataset_IDRID"
122         return {"csv": f"{base}/idrid_labels.csv", "dir": f"{base}/Imagenes/Imagenes"}
123
124     elif dataset_name == "MESSIDOR":
125         base = "/home/marian-s/Disertatie/Dataset_Messidor_2"
126         return {"csv": f"{base}/messidor_data.csv", "dir": f"{base}/messidor-2/messidor-2/preprocess"}
127
128     else:
129         raise ValueError(f"Setul de date specificat nu este recunoscut: {dataset_name}")
130
131 # =====
132 # 2. DEFINIRE CLASA DATASET
133 # =====
134 class RetinopathyDataset(Dataset):
135     def __init__(self, dataset_names, transform=None):
136         self.transform = transform
137         self.samples = []

```

```

137
138     for ds_name in dataset_names:
139         paths = get_dataset_paths(ds_name)
140         df = pd.read_csv(paths["csv"])
141
142         for idx in range(len(df)):
143             img_name = str(df.iloc[idx, 0])
144
145             # Verificare si corectie extensii fisiere
146             if not img_name.lower().endswith(('.jpg', '.jpeg', '.png')):
147                 if ds_name == "EYEPACS":
148                     img_name += '.jpeg'
149                 elif ds_name == "APTOS":
150                     img_name += '.png'
151                 else:
152                     img_name += '.jpg'
153
154             full_path = os.path.join(paths["dir"], img_name)
155             label_binar = 0 if int(df.iloc[idx, 1]) == 0 else 1
156             self.samples.append({'path': full_path, 'label': label_binar})
157
158         del df
159         gc.collect()
160
161     print(f" -> Set incarcate ({' '.join(dataset_names)}): {len(self.samples)} imagini.")
162
163     def __len__(self): return len(self.samples)
164
165     def __getitem__(self, idx):
166         path = self.samples[idx]['path']
167
168         # Fallback in cazul in care fisierul nu este gasit sub extensia implicita
169         if not os.path.exists(path):
170             baza = os.path.splitext(path)[0]
171             variante_extensii = ['.png', '.jpeg', '.jpg', '.JPG', '.PNG', '.JPEG']
172
173             for ext in variante_extensii:
174                 if os.path.exists(baza + ext):
175                     path = baza + ext
176                     break
177
178             image = Image.open(path).convert('RGB')
179             label = torch.tensor(self.samples[idx]['label'], dtype=torch.float32)
180             if self.transform: image = self.transform(image)
181             return image, label
182
183     # =====
184     # 3. INITIALIZARE MODELE
185     # =====
186     def build_model(model_name, fine_tune_deep=False):
187         if model_name == "resnet50":
188             model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V1)
189             for param in model.parameters(): param.requires_grad = False
190
191             if fine_tune_deep:
192                 print(" - Deep Fine-Tuning: Activare gradienti pentru 'layer4' (ResNet50)")
193                 for param in model.layer4.parameters(): param.requires_grad = True
194
195             num_fttrs = model.fc.in_features
196             model.fc = nn.Sequential(nn.Dropout(0.5), nn.Linear(num_fttrs, 1))
197             return model
198
199         elif model_name == "densenet121":
200             model = models.densenet121(weights=models.DenseNet121_Weights.IMAGENET1K_V1)
201             for param in model.parameters(): param.requires_grad = False
202
203             if fine_tune_deep:
204                 print(" - Deep Fine-Tuning: Activare gradienti pentru 'denseblock4' si 'norm5' (DenseNet121)")
205                 for param in model.features.denseblock4.parameters(): param.requires_grad = True
206                 for param in model.features.norm5.parameters(): param.requires_grad = True
207
208             num_fttrs = model.classifier.in_features
209             model.classifier = nn.Sequential(nn.Dropout(0.5), nn.Linear(num_fttrs, 1))
210             return model
211

```

```

212 elif model_name == "efficientnet_b0":
213     model = models.efficientnet_b0(weights=models.EfficientNet_BO_Weights.IMAGENET1K_V1)
214     for param in model.parameters(): param.requires_grad = False
215
216     if fine_tune_deep:
217         print(" - Deep Fine-Tuning: Activare gradienti pentru ultimele blocuri
convolutionale (EfficientNet-BO)")
218         for param in model.features[6].parameters(): param.requires_grad = True
219         for param in model.features[7].parameters(): param.requires_grad = True
220         for param in model.features[8].parameters(): param.requires_grad = True
221
222     num_fttrs = model.classifier[1].in_features
223     model.classifier = nn.Sequential(nn.Dropout(p=0.5, inplace=True), nn.Linear(num_fttrs,
1))
224     return model
225
226 # =====
227 # 4. EXECUTIA EXPERIMENTELOR
228 # =====
229 for idx_exp, exp in enumerate(EXPERIMENTE):
230     print(f"\n{'-'*65}")
231     print(f"Rulare Experiment {idx_exp+1}: {exp['nume']}")
232     print(f"{'-'*65}")
233
234     # 4.1 Definire pipeline de transformari
235     if exp["aug"]:
236         transforms_pipeline = transforms.Compose([
237             transforms.Resize((224, 224)),
238             transforms.RandomHorizontalFlip(), transforms.RandomVerticalFlip(),
239             transforms.RandomRotation(15), transforms.ColorJitter(brightness=0.2, contrast
=0.2),
240             transforms.ToTensor(), transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224,
0.225])
241         ])
242     else:
243         transforms_pipeline = transforms.Compose([
244             transforms.Resize((224, 224)), transforms.ToTensor(),
245             transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
246         ])
247
248     # 4.2 Initializare DataLoadere
249     print(" Pregatire set de antrenare...")
250     train_ds = RetinopathyDataset(exp["train_datasets"], transform=transforms_pipeline)
251     print(" Pregatire set de validare...")
252     test_ds = RetinopathyDataset(exp["test_datasets"], transform=transforms_pipeline)
253
254     train_loader = DataLoader(train_ds, batch_size=32, shuffle=True, num_workers=8, pin_memory
=True)
255     test_loader = DataLoader(test_ds, batch_size=32, shuffle=False, num_workers=8, pin_memory=
True)
256
257     # 4.3 Initializare Model si Optimizator
258     model = build_model(exp["model"], exp["fine_tune_deep"])
259     if torch.cuda.device_count() > 1: model = nn.DataParallel(model)
260     model = model.to(device)
261
262     params_to_train = [p for p in model.parameters() if p.requires_grad]
263     optimizer = optim.Adam(params_to_train, lr=0.001)
264
265     # 4.4 Configurare functie de Loss
266     if exp.get("use_weighted_loss", False):
267         pondere = torch.tensor([exp["pos_weight_val"]]).to(device)
268         criterion = nn.BCEWithLogitsLoss(pos_weight=pondere)
269         print(f" - Weighted Loss activat. Pondere clasa pozitiva: {exp['pos_weight_val']}")
270     else:
271         criterion = nn.BCEWithLogitsLoss()
272
273     # 4.5 Bucla de antrenare
274     istoric = {'train_loss': [], 'test_loss': [], 'test_f1': [], 'test_acc': []}
275     best_f1_score = 0.0
276
277     for epoch in range(exp["epoci"]):
278         start_time = time.time()
279
280         # Faza de antrenare
281         model.train()

```



```

282     running_train_loss = 0.0
283     for images, labels in train_loader:
284         images, labels = images.to(device), labels.to(device).unsqueeze(1)
285         optimizer.zero_grad()
286         loss = criterion(model(images), labels)
287         loss.backward()
288         optimizer.step()
289         running_train_loss += loss.item()
290
291     avg_train_loss = running_train_loss / len(train_loader)
292
293     # Faza de validare
294     model.eval()
295     running_test_loss = 0.0
296     all_preds, all_labels = [], []
297
298     with torch.no_grad():
299         for images, labels in test_loader:
300             images, labels = images.to(device), labels.to(device).unsqueeze(1)
301             outputs = model(images)
302             loss = criterion(outputs, labels)
303             running_test_loss += loss.item()
304
305             probs = torch.sigmoid(outputs)
306             all_preds.extend((probs >= 0.5).float().cpu().numpy())
307             all_labels.extend(labels.cpu().numpy())
308
309     avg_test_loss = running_test_loss / len(test_loader)
310     epoch_f1 = f1_score(all_labels, all_preds, zero_division=0)
311     epoch_acc = accuracy_score(all_labels, all_preds)
312
313     istoric['train_loss'].append(avg_train_loss)
314     istoric['test_loss'].append(avg_test_loss)
315     istoric['test_f1'].append(epoch_f1)
316     istoric['test_acc'].append(epoch_acc)
317
318     m, s = divmod(time.time() - start_time, 60)
319     print(f" Epoca [{epoch+1}/{exp['epoci']}] | Train Loss: {avg_train_loss:.4f} | Val
Loss: {avg_test_loss:.4f} | F1: {epoch_f1:.4f} | Acc: {epoch_acc*100:.1f}% | Timp: {int(m)
}m {int(s)}s", end="")
320
321     if epoch_f1 > best_f1_score:
322         best_f1_score = epoch_f1
323         torch.save(model.module.state_dict() if isinstance(model, nn.DataParallel) else
model.state_dict(), f"{exp['nume']}_best.pth")
324         print(" -> Model salvat (imbunatatire F1).")
325     else:
326         print()
327
328     del all_preds, all_labels
329     gc.collect()
330
331     # 4.6 Generare si salvare grafice
332     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))
333     ax1.plot(istoric['train_loss'], label='Train Loss', color='blue')
334     ax1.plot(istoric['test_loss'], label='Val Loss', color='red', linestyle='--')
335     ax1.set_title(f"Evolutie Loss: {exp['nume']}")
336     ax1.legend(); ax1.grid(True, alpha=0.3)
337
338     ax2.plot(istoric['test_acc'], label='Accuracy', color='green')
339     ax2.plot(istoric['test_f1'], label='F1-Score', color='purple', linestyle='-.')
340     ax2.set_title(f"Evolutie Metrici: {exp['nume']}")
341     ax2.legend(); ax2.grid(True, alpha=0.3)
342
343     plt.savefig(f"grafic_{exp['nume']}.png", dpi=300)
344     plt.close()
345
346     # 4.7 Eliberare resurse la finalul experimentului
347     print(" Finalizare experiment. Eliberare VRAM...")
348     del model, optimizer, train_loader, test_loader
349     torch.cuda.empty_cache()
350     gc.collect()
351     time.sleep(2)
352

```



```

70 ])
71
72 # Transformari pentru seturile de validare si testare
73 eval_transforms = transforms.Compose([
74     transforms.Resize((224, 224)),
75     transforms.ToTensor(),
76     transforms.Normalize(mean=[0.485, 0.456, 0.406],
77                             std=[0.229, 0.224, 0.225])
78 ])
79
80 # Citirea fisierelor CSV
81 train_df = pd.read_csv('Datasets/Dataset_APTOS_2019/train_1.csv')
82 val_df = pd.read_csv('Datasets/Dataset_APTOS_2019/valid.csv')
83 test_df = pd.read_csv('Datasets/Dataset_APTOS_2019/test.csv')
84
85 # Initializarea obiectelor de tip Dataset
86 train_dataset = DiabeticRetinopathyDataset(train_df, 'Datasets/Dataset_APTOS_2019/train_images/
/train_images', transform=train_transforms)
87 val_dataset = DiabeticRetinopathyDataset(val_df, 'Datasets/Dataset_APTOS_2019/val_images/
val_images', transform=eval_transforms)
88 test_dataset = DiabeticRetinopathyDataset(test_df, 'Datasets/Dataset_APTOS_2019/test_images/
test_images', transform=eval_transforms)
89
90 # Initializarea obiectelor de tip DataLoader
91 train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True, num_workers=8)
92 val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False, num_workers=8)
93 test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False, num_workers=8)
94
95 print(f"Set de antrenare: {len(train_dataset)} imagini | Set de validare: {len(val_dataset)}
imagini | Set de testare: {len(test_dataset)} imagini")
96
97 import torchvision.models as models
98 import torch.nn as nn
99
100 def create_model(num_classes=5):
101     # Incarcarea arhitecturii ResNet50 pre-antrenata pe ImageNet
102     model = models.resnet50(weights=models.ResNet50_Weights.DEFAULT)
103
104     # Inghetarea parametrilor straturilor convolutionale de baza
105     for param in model.parameters():
106         param.requires_grad = False
107
108     # Modificarea stratului Fully Connected (FC) pentru clasificare in 5 clase
109     num_fters = model.fc.in_features
110     model.fc = nn.Sequential(
111         nn.Linear(num_fters, 512),
112         nn.ReLU(),
113         nn.Dropout(0.4),
114         nn.Linear(512, num_classes)
115     )
116
117     # Activarea gradientilor pentru noul strat FC
118     for param in model.fc.parameters():
119         param.requires_grad = True
120
121     return model
122
123 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
124 model = create_model().to(device)
125 print(f"Dispozitiv de rulare configurat: {device}")
126
127 # Importuri aditionale pentru bucla de antrenare
128 from sklearn.model_selection import train_test_split
129 import torch.optim as optim
130 from tqdm import tqdm
131
132 # Definirea functiei de cost si a optimizatorului
133 criterion = nn.CrossEntropyLoss()
134 optimizer = optim.Adam(model.fc.parameters(), lr=0.0001)
135
136 EPOCHS = 100
137 best_val_acc = 0.0
138 train_losses = []
139 val_losses = []
140 train_accuracies = []
141 val_accuracies = []

```

```

142
143 # BUCLA DE ANTRENARE SI VALIDARE
144 for epoch in range(EPOCHS):
145     # --- Faza de Antrenare ---
146     model.train()
147     running_loss, correct, total = 0.0, 0, 0
148
149     print(f"\nEpoca {epoch+1}/{EPOCHS}")
150     for images, labels in tqdm(train_loader, desc="Training"):
151         images, labels = images.to(device), labels.to(device)
152
153         optimizer.zero_grad()
154         outputs = model(images)
155         loss = criterion(outputs, labels)
156         loss.backward()
157         optimizer.step()
158
159         running_loss += loss.item()
160         _, predicted = torch.max(outputs.data, 1)
161         total += labels.size(0)
162         correct += (predicted == labels).sum().item()
163
164     train_acc = 100 * correct / total
165     train_accuracies.append(train_acc)
166     train_loss = running_loss / len(train_loader)
167     train_losses.append(train_loss)
168
169     # --- Faza de Validare ---
170     model.eval()
171     val_loss, val_correct, val_total = 0.0, 0, 0
172
173     with torch.no_grad():
174         for images, labels in val_loader:
175             images, labels = images.to(device), labels.to(device)
176             outputs = model(images)
177             loss = criterion(outputs, labels)
178
179             val_loss += loss.item()
180             _, predicted = torch.max(outputs.data, 1)
181             val_total += labels.size(0)
182             val_correct += (predicted == labels).sum().item()
183
184     val_acc = 100 * val_correct / val_total
185     val_loss = val_loss / len(val_loader)
186
187     # Adaugare in lista a valorilor de validare o singura data pe epoca
188     val_accuracies.append(val_acc)
189     val_losses.append(val_loss)
190
191     print(f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.2f}%")
192     print(f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.2f}%")
193
194     # Salvarea modelului optim pe baza acuratetii de validare
195     if val_acc > best_val_acc:
196         best_val_acc = val_acc
197         torch.save(model.state_dict(), 'best_dr_model.pth')
198         print("-> Model salvat (imbunatatire performanta validare).")
199
200
201 from sklearn.metrics import confusion_matrix, classification_report, cohen_kappa_score
202 import numpy as np
203
204 print("\n--- FAZA DE TESTARE ---")
205
206 # Incarcarea ponderilor modelului optim
207 model.load_state_dict(torch.load('best_dr_model.pth'))
208 model.eval()
209
210 test_correct = 0
211 test_total = 0
212 test_preds = []
213 test_labels = []
214
215 with torch.no_grad():
216     for images, labels in tqdm(test_loader, desc="Testing"):
217         images, labels = images.to(device), labels.to(device)

```

```

218         outputs = model(images)
219
220         _, predicted = torch.max(outputs.data, 1)
221
222         test_total += labels.size(0)
223         test_correct += (predicted == labels).sum().item()
224
225         test_preds.extend(predicted.cpu().numpy())
226         test_labels.extend(labels.cpu().numpy())
227
228     test_acc = 100 * test_correct / test_total
229     print(f"\nAcuratete finala pe setul de testare: {test_acc:.2f}%\n")
230
231     # Calculul metricii Quadratic Weighted Kappa (QWK)
232     qwk = cohen_kappa_score(test_labels, test_preds, weights='quadratic')
233     print(f"Scor Quadratic Weighted Kappa (QWK): {qwk:.4f}")
234     # Un scor > 0.7 indica un acord bun in aplicatiile de imagistica medicala
235
236     # Generarea matricei de confuzie
237     cm = confusion_matrix(test_labels, test_preds)
238     plt.figure(figsize=(8,6))
239     sns.heatmap(cm, annot=True, fmt='d', cmap='Oranges',
240                 xticklabels=['Fara', 'Usor', 'Moderat', 'Sever', 'Proliferativ'],
241                 yticklabels=['Fara', 'Usor', 'Moderat', 'Sever', 'Proliferativ'])
242     plt.xlabel('Predictie Model')
243     plt.ylabel('Eticheta Reala')
244     plt.title(f'Matrice de Confuzie - Set de Testare (Acuratete: {test_acc:.2f}%')')
245     plt.show()
246
247     # Generarea raportului de clasificare (Precision, Recall, F1-Score)
248     print("\nRaport de clasificare:")
249     print(classification_report(test_labels, test_preds, target_names=['Fara (0)', 'Usor (1)', 'Moderat (2)', 'Sever (3)', 'Proliferativ (4)']))
250
251     # Generarea graficelor pentru curbele de invatare
252     plt.figure(figsize=(12, 5))
253     plt.subplot(1, 2, 1)
254     plt.plot(train_losses, label='Train Loss')
255     plt.plot(val_losses, label='Validation Loss')
256     plt.title('Evolutie Loss')
257     plt.xlabel('Epoca')
258     plt.ylabel('Loss')
259     plt.legend()
260
261     plt.subplot(1, 2, 2)
262     plt.plot(train_accuracies, label='Train Accuracy')
263     plt.plot(val_accuracies, label='Validation Accuracy')
264     plt.title('Evolutie Acuratete')
265     plt.xlabel('Epoca')
266     plt.ylabel('Acuratete (%)')
267     plt.legend()
268     plt.show()

```

Listing A.2: Second try